

DATA STRUCTURES & ALGORITHMS (DSA)

LAB PROJECT REPORT

Project Title

MazeSolver – Maze Generation and Solving using DFS and BFS

Submitted By

Umme Ruman

Registration No: 20241-36852

Rabeya Asif

Registration No: 20241-35761

Course Information

- **Course:** Data Structures & Algorithms (DSA) Lab
 - **Semester:** 3rd Semester (Software Engineering)
-

Instructor

Ms. Sahifa Hashmi

Institution

Department of Software Engineering IOBM

1. Introduction

Maze solving is a common problem used to understand graph traversal algorithms in Data Structures and Algorithms. In this project, we designed and implemented a **MazeSolver** using **Depth First Search (DFS)** for maze generation and **Breadth First Search (BFS)** for finding the shortest path from start to end. The project is implemented in Java and visualized using the **StdDraw** library.

2. Objectives of the Project

- To implement DFS for random maze generation
 - To implement BFS for maze solving
 - To apply stack and queue data structures
 - To visualize traversal and solution paths
-

3. Tools and Technologies Used

- Programming Language: Java
 - IDE: Visual Studio Code / IntelliJ IDEA
 - Library: StdDraw
 - Concepts: DFS, BFS, Stack, Queue, Graph Traversal
-

4. Project Overview

The project consists of two main files:

1. **MazeSolver.java** – Contains maze generation, maze solving, and logic implementation
2. **StdDraw.java** – Used for graphical visualization of the maze

The maze is represented as a grid where each cell has four possible walls: left, right, up, and down. These walls are removed to create paths.

5. Data Structures Used

- **Stack:** Used in DFS for maze generation
 - **Queue:** Used in BFS for maze solving
 - **LinkedList:** Used to store paths
 - **HashMap:** Used to store parent relationships
 - **2D Arrays:** Used for maze grid and visited tracking
-

6. Maze Generation using DFS (Depth First Search)

Theory (DFS)

Depth First Search is a graph traversal algorithm that explores as far as possible along one branch before backtracking. It uses a **Stack** data structure.

Implementation in Project

- A stack is used to keep track of cells
- Starting from (0,0), the algorithm visits unvisited neighboring cells
- Walls between cells are removed to create paths
- Backtracking occurs when no unvisited neighbors are found

Why DFS for Maze Generation?

- Produces a perfect maze (no loops, only one path between any two cells)
 - Easy to implement using stacks
-

7. Maze Solving using BFS (Breadth First Search)

Theory (BFS)

Breadth First Search explores all neighboring nodes level by level. It uses a **Queue** data structure and guarantees the shortest path in an unweighted graph.

Implementation in Project

- A queue is used to explore cells
- Parent mapping is stored using a HashMap
- Once the destination is reached, the path is reconstructed
- BFS ensures the shortest path from start to end

Why BFS for Maze Solving?

- Guarantees shortest path
 - Efficient and reliable for grid-based problems
-

8. Visualization using StdDraw

The **StdDraw** library is used to:

- Draw maze walls
- Display visited cells in **gray color**
- Display solution path in **blue color**
- Mark start cell in **green**
- Mark end cell in **red**

This visual output makes the algorithm easier to understand and analyze.

9. Algorithm Complexity

- **DFS Maze Generation:** Time $O(N^2)$, Space $O(N^2)$
 - **BFS Maze Solving:** Time $O(N^2)$, Space $O(N^2)$
-

CODE:

```
import java.util.*;
```

```
public class Mazesolver {
```

```
// ===== ENUM =====
```

```
public enum Direction { LEFT, RIGHT, UP, DOWN }
```

```
// ===== CELL =====
```

```
public static class Cell {
```

```
    int x, y;
```

```
    Cell(int x, int y) { this.x = x; this.y = y; }
```

```
    public void draw(double r, java.awt.Color c) {
```

```
        StdDraw.setPenColor(c);
```

```
        StdDraw.filledCircle(x + 0.5, y + 0.5, r);
```

```
    }
```

```
    @Override
```

```
    public boolean equals(Object o) {
```

```
        if (!(o instanceof Cell)) return false;
```

```
        Cell c = (Cell) o;
```

```
        return x == c.x && y == c.y;
```

```
    }
```

```
    @Override
```

```
    public int hashCode() { return Objects.hash(x, y); }
```

```
}
```

```
// ===== PATH =====
```

```
public static class Path implements Iterable<Cell> {
```

```
    LinkedList<Cell> path = new LinkedList<>();
```

```
    public void add(Cell c) { path.add(c); }
```

```
public Iterator<Cell> iterator() { return path.iterator(); }
```

```
}
```

```
// ===== MAZE CLASS =====
```

```
public static class Maze {
```

```
    int size;
```

```
    boolean[][] visited;
```

```
    boolean[][] left, right, up, down;
```

```
    Cell start, end;
```

```
    Path visitedPath = new Path();
```

```
    Path solutionPath = null;
```

```
    public Maze(int size) {
```

```
        this.size = size;
```

```
        visited = new boolean[size][size];
```

```
        left = new boolean[size][size];
```

```
        right = new boolean[size][size];
```

```
        up = new boolean[size][size];
```

```
        down = new boolean[size][size];
```

```
        // initially all walls present
```

```
        for (int i=0;i<size;i++) {
```

```
            for (int j=0;j<size;j++) {
```

```
                left[i][j] = right[i][j] = up[i][j] = down[i][j] = true;
```

```
    }
```

```
    }
```

```
}
```

```
    public void setStart(int x, int y) { start = new Cell(x,y); }
```

```
    public void setEnd(int x, int y) { end = new Cell(x,y); }
```

```
    public void removeWall(int x,int y, Direction dir) {
```

```
        switch(dir){
```

```
            case LEFT:
```

```
                left[x][y]=false; right[x-1][y]=false; break;
```

```
            case RIGHT:
```

```
                right[x][y]=false; left[x+1][y]=false; break;
```

```
            case UP:
```

```
                up[x][y]=false; down[x][y+1]=false; break;
```

```
            case DOWN:
```

```
                down[x][y]=false; up[x][y-1]=false; break;
```

```
        }
```

```
    }
```

```
    public boolean isOpen(int x, int y, Direction d){
```

```
        switch(d){
```

```
            case LEFT: return !left[x][y];
```

```
            case RIGHT: return !right[x][y];
```

```
        case UP: return !up[x][y];
```

```
        case DOWN: return !down[x][y];
```

```
    }
```

```
    return false;
```

```
}
```

```
public void visit(int x,int y){
```

```
    visited[x][y] = true;
```

```
    visitedPath.add(new Cell(x,y));
```

```
}
```

```
public boolean isVisited(int x,int y){ return visited[x][y]; }
```

```
public void setSolution(Path p){ solutionPath = p; }
```

```
// DRAWING
```

```
public void draw() {
```

```
    StdDraw.clear();
```

```
    StdDraw.setXscale(0,size);
```

```
    StdDraw.setYscale(0,size);
```

```
// draw maze walls
```

```
StdDraw.setPenColor(StdDraw.BLACK);
```

```
for(int x=0;x<size;x++){
```

```
for(int y=0;y<size;y++){
```

```
    if(left[x][y]) StdDraw.line(x, y, x, y+1);
```

```
    if(right[x][y]) StdDraw.line(x+1, y, x+1, y+1);
```

```
    if(up[x][y]) StdDraw.line(x, y+1, x+1, y+1);
```

```
    if(down[x][y]) StdDraw.line(x, y, x+1, y);
```

```
}
```

```
}
```

```
// draw visited (grey)
```

```
for(Cell c: visitedPath.path){
```

```
    if(!c.equals(start) && !c.equals(end))
```

```
        c.draw(0.35, java.awt.Color.LIGHT_GRAY);
```

```
}
```

```
// draw solution (blue)
```

```
if(solutionPath != null){
```

```
    for(Cell c: solutionPath.path){
```

```
        if(!c.equals(start) && !c.equals(end))
```

```
            c.draw(0.35, java.awt.Color.BLUE);
```

```
}
```

```
}
```

```
// draw start (green)
```

```
if(start != null) start.draw(0.45, java.awt.Color.GREEN);
```

```
// draw end (red)
```

```
if(end != null) end.draw(0.45, java.awt.Color.RED);
```

```
StdDraw.show();
```

```
}
```

```
}
```

```
// ===== MAZE GENERATOR (DFS) =====
```

```
public static class MazeGenerator {
```

```
    Random rand = new Random();
```

```
    public Maze generate(int size){
```

```
        Maze m = new Maze(size);
```

```
        boolean[][] visited = new boolean[size][size];
```

```
        Stack<Cell> stack = new Stack<>();
```

```
        stack.push(new Cell(0,0));
```

```
        visited[0][0] = true;
```

```
        while(!stack.isEmpty()){
```

```
            Cell c = stack.peek();
```

```
            int x=c.x, y=c.y;
```

```
ArrayList<Direction> dirs = new ArrayList<>();
```

```
if(x>0 && !visited[x-1][y]) dirs.add(Direction.LEFT);
```

```
if(x<size-1 && !visited[x+1][y]) dirs.add(Direction.RIGHT);
```

```
if(y>0 && !visited[x][y-1]) dirs.add(Direction.DOWN);
```

```
if(y<size-1 && !visited[x][y+1]) dirs.add(Direction.UP);
```

```
if(dirs.isEmpty()) { stack.pop(); continue; }
```

```
Direction d = dirs.get(rand.nextInt(dirs.size()));
```

```
m.removeWall(x,y,d);
```

```
switch(d){
```

```
    case LEFT: x--; break;
```

```
    case RIGHT: x++; break;
```

```
    case DOWN: y--; break;
```

```
    case UP: y++; break;
```

```
}
```

```
visited[x][y] = true;
```

```
stack.push(new Cell(x,y));
```

```
}
```

```
// start & end random
```

```
m.setStart(0,0);
```

```
m.setEnd(size-1,size-1);
```

```
return m;
```

```
}
```

```
}
```

```
// ===== BFS SOLVER =====
```

```
public static Path solve(Maze m){
```

```
    Queue<Cell> q = new LinkedList<>();
```

```
    Map<Cell,Cell> parent = new HashMap<>();
```

```
    q.add(m.start);
```

```
    m.visit(m.start.x, m.start.y);
```

```
    while(!q.isEmpty()){
```

```
        Cell c = q.remove();
```

```
        if(c.equals(m.end)) break;
```

```
        int x=c.x, y=c.y;
```

```
        if(x>0 && !m.isVisited(x-1,y) && m.isOpen(x,y,Direction.LEFT)) {
```

```
            m.visit(x-1,y);
```

```
            Cell n=new Cell(x-1,y);
```

```
parent.put(n,c);
```

```
q.add(n);
```

```
}
```

```
if(x<m.size-1 && !m.isVisited(x+1,y) && m.isOpen(x,y,Direction.RIGHT)) {
```

```
    m.visit(x+1,y);
```

```
    Cell n=new Cell(x+1,y);
```

```
    parent.put(n,c);
```

```
    q.add(n);
```

```
}
```

```
if(y>0 && !m.isVisited(x,y-1) && m.isOpen(x,y,Direction.DOWN)) {
```

```
    m.visit(x,y-1);
```

```
    Cell n=new Cell(x,y-1);
```

```
    parent.put(n,c);
```

```
    q.add(n);
```

```
}
```

```
if(y<m.size-1 && !m.isVisited(x,y+1) && m.isOpen(x,y,Direction.UP)) {
```

```
    m.visit(x,y+1);
```

```
    Cell n=new Cell(x,y+1);
```

```
    parent.put(n,c);
```

```
    q.add(n);
```

```
}
```

```
}
```

```
// reconstruct path
```

```
Path p = new Path();
```

```
Cell curr = m.end;
```

```
while(curr != null && !curr.equals(m.start)){
```

```
    p.path.addFirst(curr);
```

```
    curr = parent.get(curr);
```

```
}
```

```
p.path.addFirst(m.start);
```

```
return p;
```

```
}
```

```
// ===== MAIN =====
```

```
public static void main(String[] args) {
```

```
    StdDraw.enableDoubleBuffering();
```

```
    int size = 20;
```

```
    MazeGenerator gen = new MazeGenerator();
```

```
    Maze maze = gen.generate(size);
```

```
Path sol = solve(maze);
```

```
maze.setSolution(sol);
```

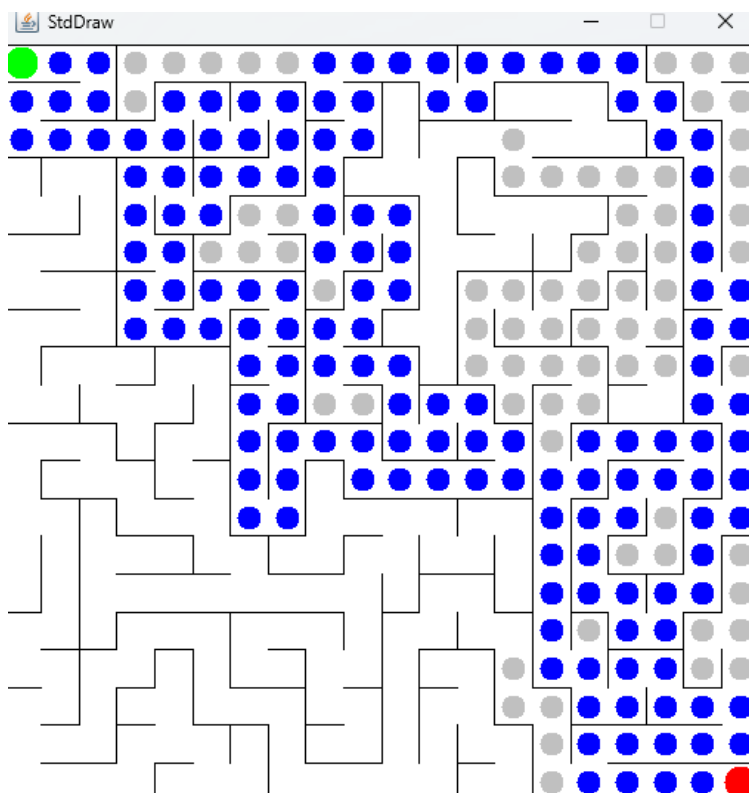
```
maze.draw();
```

```
}
```

```
}
```

10. Output Description

- A randomly generated maze of size 20x20
- Gray cells show all visited nodes
- Blue cells show the shortest solution path
- Green circle indicates start
- Red circle indicates destination



11. Learning Outcomes

- Clear understanding of DFS and BFS
 - Practical use of stacks and queues
 - Improved problem-solving skills
 - Experience with graphical visualization in Java
-

12. Conclusion

This project demonstrates the practical implementation of DFS and BFS in solving real-world problems. DFS efficiently generates the maze, while BFS guarantees the shortest path solution. The MazeSolver project strengthened our understanding of core DSA concepts and Java programming.
